

Optimization and Simulation for Industrial Automation

Final project

Profs. Alberto Tarable, Marco Ghirardi

2025/2026

-
- The project shall be implemented by using Matlab/Simulink. Source code / Simulink model must be delivered along with a short explanatory report.
 - The delivery of the project should be done before the exam session you subscribe.
 - Maximum grade: 5 points if you deliver before the first Winter exam session, 4 points otherwise.
 - The evaluation takes into account the following characteristics:
 - correspondence to the description
 - correctness (the program must run and give the expected results)
 - organization of the program
 - completeness of the implementation
 - quality of the report
-

A small language model (SLM) is implemented with two CPUs and two GPUs, according to the architecture of Figure 1.

- The user prompts arrive to CPU 1 for a first processing.
- At the output of CPU 1, prompts are sent to one of the two GPUs, according to a given policy, for inference.
- After GPU processing, prompts enter CPU 2 for a final refinement.
- The SLM answer is then submitted to the user, which can either accept the output with probability p_A , or require a better answer with probability $1 - p_A$. In the former case, the prompt leaves the system, while in the latter it re-enters the system at CPU 1 for a new cycle.

CPU 1 can be modelled as a M/M/1 queue, while all the other system components can be represented as M/M/1/K queues, with a single server and a finite capacity. The service times are exponentially distributed with rates $\mu_{C,1} = \mu_{C,2} = 8$ prompts/minute for the two CPUs, and $\mu_{G,1} = \mu_{G,2} = 5$ prompts/minute for the two GPUs. The capacity of CPU 2 is denoted by K_C , while the capacities of the two GPUs are equal and denoted by K_G . (Notice that the capacity of a M/M/1/K queue includes the customer/prompt being served.)

1 Human user scenario

In the first scenario, prompts arrive from human users according to a Poisson process with rate $\gamma_P = 5$ prompts/minute (i.e., the interarrival times are i.i.d. exponential random variables with mean $1/\gamma_P$) and are served according to FIFO policy. Also $p_A = 0.7$.

1. Implement the model in Simulink, with parameters K_C and K_G .

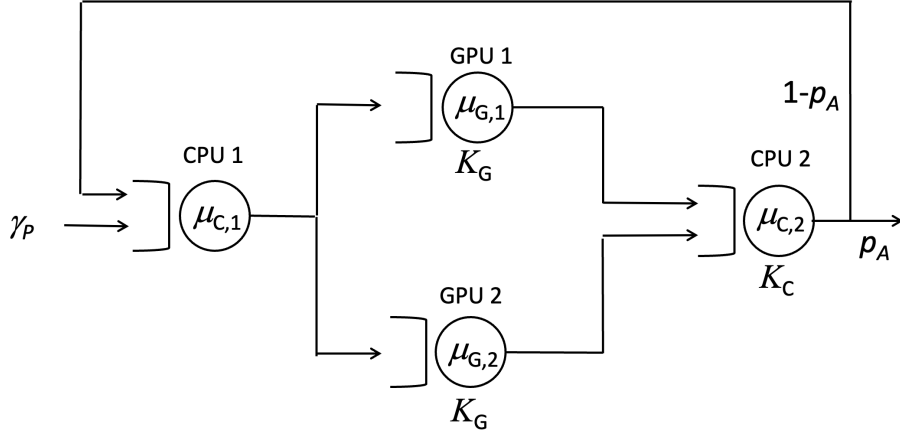


Figura 1: The SLM model.

2. Through simulation, design the minimum capacity values for the queues so that the network is ergodic, in the following three conditions:

- Prompts are distributed to the two GPUs with uniform probabilities (on the average, 50 % of the prompts go to GPU 1, 50 % to GPU 2), irrespectively of their current load of prompts. If the chosen GPU is full, the prompt waits in CPU 1.
- Prompts are sent to the GPU with the smallest current load (number of prompts in the GPU). If the GPUs have the same load, the destination GPU is chosen at random. As in the previous case, if the chosen GPU is full, the prompt waits in CPU 1.
- The two GPUs are unified in a single M/M/2/K queue with capacity $2K_G$. If such queue is full, the prompt waits in CPU 1.

2 Agent scenario

In the second scenario, an AI tool (agent) organises autonomously a trip, a conference, a wedding, etc., by submitting to the SLM a batch of N parallel prompts. The SLM manages efficiently the resources by optimising the order of service and the assignment to the GPUs of the single prompts in the batch. We consider here a particular configuration of the system:

- All N prompts are available in the buffer of CPU 1 at time 0. The processing times needed by each prompt on each CPU/GPU is already known at time 0 (they can be extracted from their distribution when the prompt is created). Note that the service times of GPU 1 and GPU 2 are different extractions.
- Buffers can be considered infinite ($K_G = K_C = \infty$).
- Acceptance probability is $p_A = 1$.
- The buffer of CPU 1 is random access.
- The buffers of GPU 1, GPU 2 and CPU 2 are FIFO.

Implement the model in Simulink, considering the following variants:

1. Prompts are sent to CPU 1 in random order (you can simply consider the index order).
2. A linear programming model of the problem is run at time 0, and provides the best ordering of the operations on CPU 1 and assignments of each prompt to GPU 1 or GPU 2.

Compare the results of the two variants for $N = 5$, $N = 7$ and $N = 9$. The optimization objective function is the makespan of all the operations (minimization of the maximum completion time).

Suggestions

- In the first scenario, to get the number of entities in the queue block, directly connect the corresponding statistics output to a Simulink function block implementing the identity function $y = x$.
 - For the agent scenario, when N increases, if the model is not efficient the solver may take too much time for obtaining the optimal solution. If you want to set a time limit to the solver execution (the output solution, in this case, may be not yet optimal) use the following functions when running the solver to set a limit of 60 seconds:
 - `opt = optimoptions('intlinprog','MaxTime',60);`
 - `[Sol fval exitflag]=solve(problem,'solver','intlinprog', Options=opt);`
 - The processing times in the agent scenario are extracted from their distribution when the prompts are created. Set a seed in order to be able to repeat the experiments on the same data with and without optimizer. Perform the comparison using an average over instances obtained with 5 different seeds.
-